

# Algorithms 2

## Reductions

Daniel Rosenberg & Michael Trushkin

Ariel University

# 1 NP-completeness

---

# 1.1 NP-completeness

1 NP-completeness

- As we saw, most problems are *hard*!

- As we saw, most problems are *hard*!

**Definition 1.1.1 (NP class)** A language  $L$  is said to be in **NP** if we have a polynomial-time algorithm  $M$  such that

$$x \in L \Leftrightarrow \exists y \text{ s.t. } |y| < p(|x|) \text{ and } M(x, y) = 1$$

where  $p$  is some polynomial

- In most literature  $y$  is called a *witness* and  $M$  is called *verifying algorithm*, where  $y$  plays the role of the answer, and  $M$  should just verify if the answer is correct.

## 1.1 NP-completeness

- As we saw, most problems are *hard*!

**Definition 1.1.1 (NP class)** A language  $L$  is said to be in **NP** if we have a polynomial-time algorithm  $M$  such that

$$x \in L \Leftrightarrow \exists y \text{ s.t. } |y| < p(|x|) \text{ and } M(x, y) = 1$$

where  $p$  is some polynomial

- In most literature  $y$  is called a *witness* and  $M$  is called *verifying algorithm*, where  $y$  plays the role of the answer, and  $M$  should just verify if the answer is correct.
- We talked about a few:
  - $k$ -clique
  - CNF-SAT
  - $k$ -CNF-SAT

## 2 Reductions

---

## 2.1 Reductions

- We saw we can “translate” one language into another with reductions

## 2.1 Reductions

- We saw we can “translate” one language into another with reductions

**Definition 2.1.1 (polynomial time reduction)** Given two languages  $L_1, L_2 \in \mathbf{NP}$ , we write  $L_1 \leq_p L_2$  if there exists a function  $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$  and a polynomial  $p : \mathbb{N} \rightarrow \mathbb{N}$ , such that:

- $x \in L_1 \Leftrightarrow f(x) \in L_2$
- for every  $x \in \{0, 1\}^*$ ,  $f$  runs in  $p(|x|)$  time.

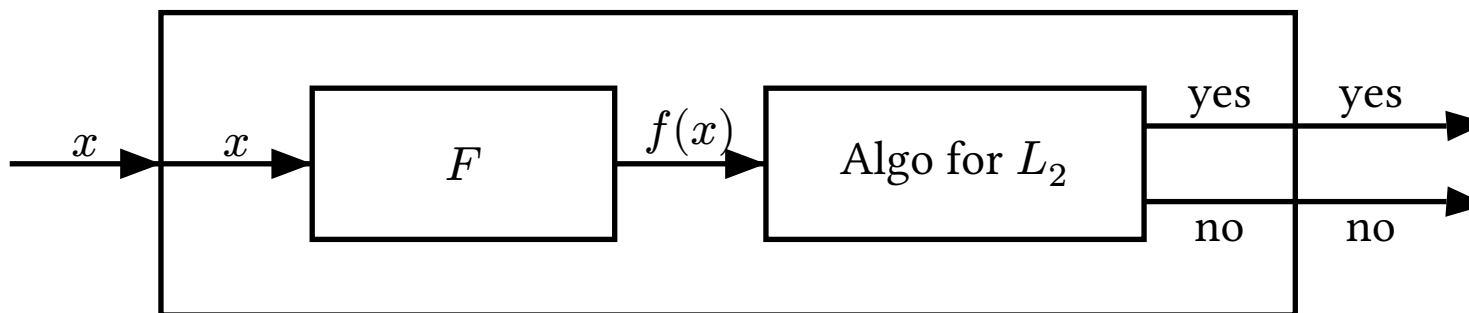


## 2.1 Reductions

- We saw we can “translate” one language into another with reductions

**Definition 2.1.1 (polynomial time reduction)** Given two languages  $L_1, L_2 \in \mathbf{NP}$ , we write  $L_1 \leq_p L_2$  if there exists a function  $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$  and a polynomial  $p : \mathbb{N} \rightarrow \mathbb{N}$ , such that:

- $x \in L_1 \Leftrightarrow f(x) \in L_2$
- for every  $x \in \{0, 1\}^*$ ,  $f$  runs in  $p(|x|)$  time.



## 2.2 Hard Languages

- For any complexity class, there are hard languages

**Definition 2.2.1 (NP-hard)** A language  $L \subseteq \{0, 1\}^*$  is said to be NP-hard if  $L' \leq_p L$  for every  $L' \in \mathbf{NP}$

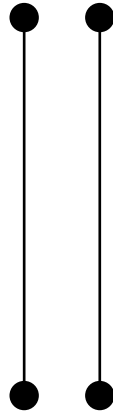
**Definition 2.2.2 (NP-complete)** A language  $L \subseteq \{0, 1\}^*$  is said to be NP-complete if  $L \in \mathbf{NP}$  and  $L$  is NP-hard

- We saw that CNF-SAT and  $k$ -CNF-SAT are NP Complete

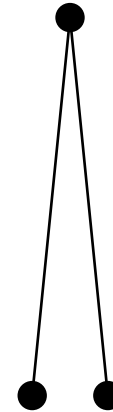
## 3 Independent set

---

For a graph  $G$ , two edges  $e_1, e_2 \subseteq E(G)$  are called *independent* if there is no common vertex between them.



independent



not independent

### 3.1 Independent set

- For a graph  $G$ , let  $\alpha(G)$  denote the size of the maximum independent set in  $G$ .

**Definition 3.1.1**  $\text{IS} := \{ \langle G, k \rangle : \alpha(G) \geq k \}$ .

**Theorem 3.1.2** IS is in NPC.

## 3.2 Independent set

- The proof that IS is in **NP** is left as homework.
- In order to show that IS hard we need to show that

$$L \leq_p \text{IS}$$

for some **NP** hard language.

- Here, we show that

$$\text{3-CNF-SAT} \leq_p \text{IS}$$

### 3.3 3-CNF-SAT $\leq_p$ IS

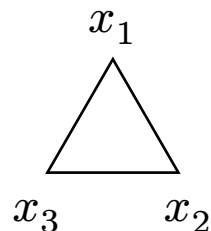
- Given a 3-CNF formula  $\varphi$ , construct a graph  $G_\varphi$  as follows:
  - Triangles:** For each clause  $l_1, l_2, l_3$  we create a triangle with 3 vertices named  $v_{l_1}, v_{l_2}, v_{l_3}$ .
  - Consistency Edges:** For any pair of complementary literals  $x_j, \overline{x_j}$  that are in different clauses, put an edge between the vertices that correspond to the literals.

$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$

### 3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula  $\varphi$ , construct a graph  $G_\varphi$  as follows:
  - Triangles:** For each clause  $l_1, l_2, l_3$  we create a triangle with 3 vertices named  $v_{l_1}, v_{l_2}, v_{l_3}$ .
  - Consistency Edges:** For any pair of complementary literals  $x_j, \overline{x_j}$  that are in different clauses, put an edge between the vertices that correspond to the literals.

$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$

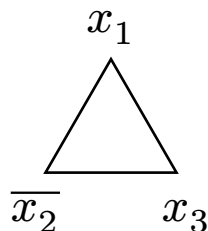
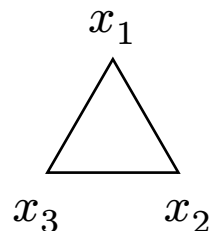




### 3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula  $\varphi$ , construct a graph  $G_\varphi$  as follows:
  - Triangles:** For each clause  $l_1, l_2, l_3$  we create a triangle with 3 vertices named  $v_{l_1}, v_{l_2}, v_{l_3}$ .
  - Consistency Edges:** For any pair of complementary literals  $x_j, \overline{x_j}$  that are in different clauses, put an edge between the vertices that correspond to the literals.

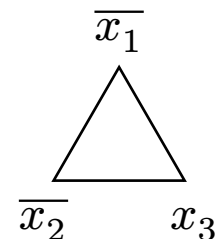
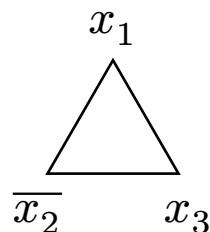
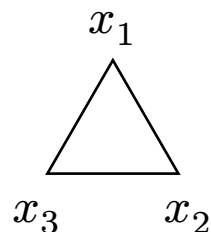
$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



### 3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula  $\varphi$ , construct a graph  $G_\varphi$  as follows:
  - Triangles:** For each clause  $l_1, l_2, l_3$  we create a triangle with 3 vertices named  $v_{l_1}, v_{l_2}, v_{l_3}$ .
  - Consistency Edges:** For any pair of complementary literals  $x_j, \overline{x_j}$  that are in different clauses, put an edge between the vertices that correspond to the literals.

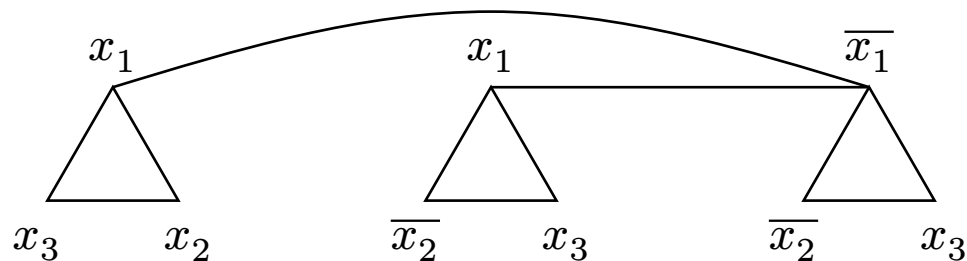
$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



### 3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula  $\varphi$ , construct a graph  $G_\varphi$  as follows:
  - Triangles:** For each clause  $l_1, l_2, l_3$  we create a triangle with 3 vertices named  $v_{l_1}, v_{l_2}, v_{l_3}$ .
  - Consistency Edges:** For any pair of complementary literals  $x_j, \overline{x_j}$  that are in different clauses, put an edge between the vertices that correspond to the literals.

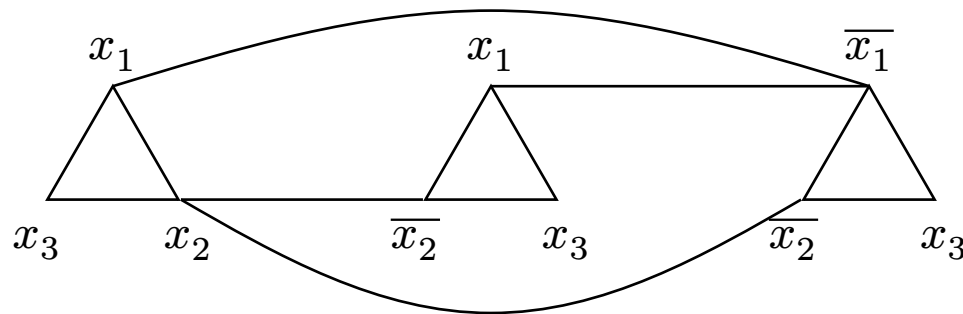
$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



### 3.3 3-CNF-SAT $\leq_p$ IS

- Given a 3-CNF formula  $\varphi$ , construct a graph  $G_\varphi$  as follows:
  - Triangles:** For each clause  $l_1, l_2, l_3$  we create a triangle with 3 vertices named  $v_{l_1}, v_{l_2}, v_{l_3}$ .
  - Consistency Edges:** For any pair of complementary literals  $x_j, \overline{x_j}$  that are in different clauses, put an edge between the vertices that correspond to the literals.

$$\varphi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee x_3)$$



- return the pair  $\langle G_\varphi, m \rangle$  where  $m$  is the number of clauses.

### 3.4 Independent set

- The algorithm runs in poly time(why?)
- Now we need to prove that

$$\varphi \in \text{3-CNF-SAT} \iff \langle G_\varphi, m \rangle \in \text{IS}$$

$\implies$ :

- Let  $\varphi$  be satisfiable and let  $\alpha_\varphi$  be a satisfying assignment for  $\varphi$ .
- As  $\alpha_\varphi$  satisfies  $\varphi$ , at least one literal of each clause is satisfied, pick any one such literal from each clause.
- The set of vertices corresponding to the set of literals chosen is independent in  $G_\varphi$  and has size of  $m$ .

### 3.5 Independent set

- The algorithm runs in poly time(why?)
- Now we need to prove that

$$\varphi \in \text{3-CNF-SAT} \iff \langle G_\varphi, m \rangle \in \text{IS}$$

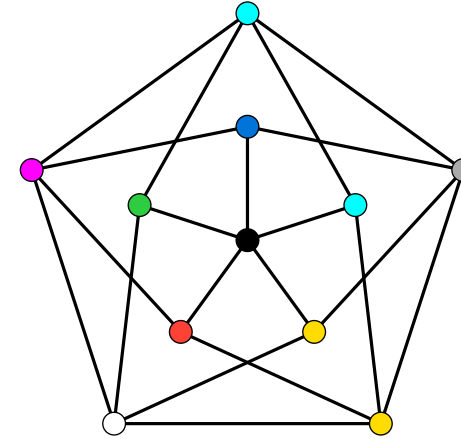
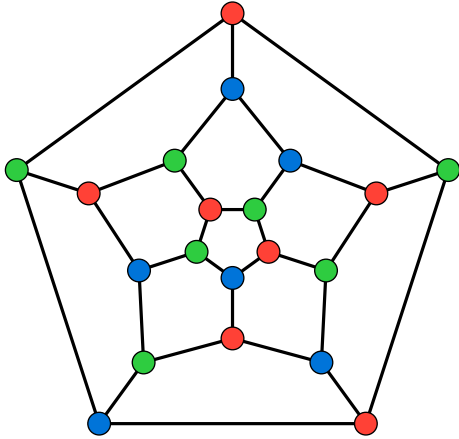
$\Leftarrow$ :

- Suppose  $G_\varphi$  has an independent set of size  $m$ , and let  $S$  be such an independent set.
- Let  $l_1, \dots, l_m$  denote the vertices of  $S$ ,
- for each  $i \in m$  let  $v(l_i) \in \{x_1, \overline{x_1}, \dots, x_n, \overline{x_n}\}$  be the variable corresponding to  $l_i$ .
- Set  $v(l_i) = T$  for all  $i \in [m]$ , Finally if  $x_i \in [n]$  did not receive an assignment set  $x_i = T$ .
- The assignment is satisfying:
  - every triangle has one vertex in  $S \Rightarrow$  every clause has one positive literal
- The assignment is *consistent*:
  - if some variable  $v(l_i) = v(\overline{l_i}) \Rightarrow$  both end of  $(l_i, \overline{l_i})$  are in  $S$ .

## 4 Graph coloring

---

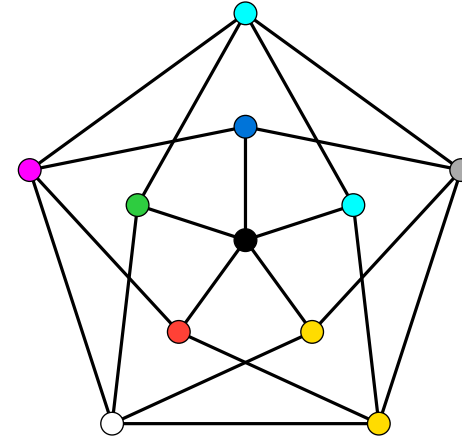
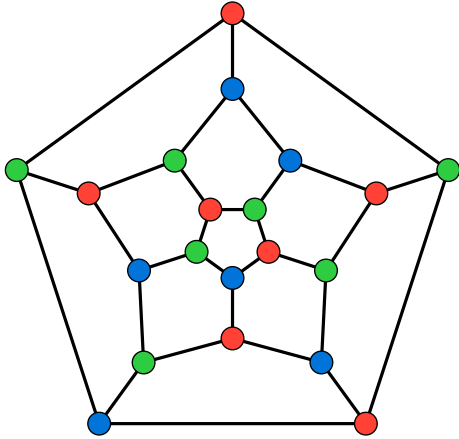
## 4.1 Graph coloring



- For a graph  $G$  denote by  $\chi(G)$  the least  $k \in \mathbb{N}$  such that  $G$  is  $k$ -colorable.



# 4.1 Graph coloring

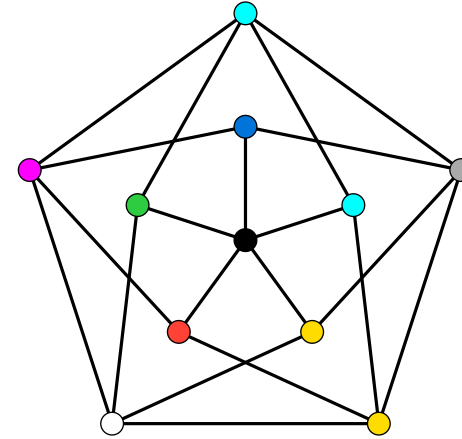
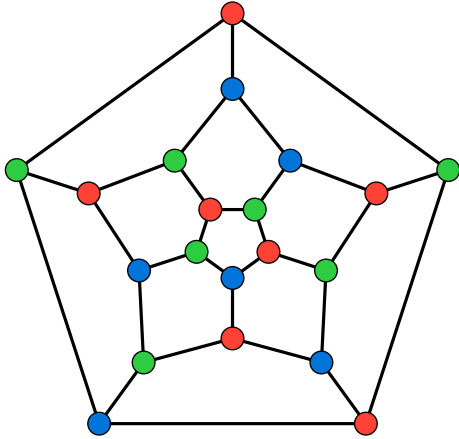


- For a graph  $G$  denote by  $\chi(G)$  the least  $k \in \mathbb{N}$  such that  $G$  is  $k$ -colorable.

**Definition 4.1.1**  $k\text{-COL} := \{G : \chi(G) \leq k\}$

It is well known that  $2\text{-COL} \in \mathbf{P}$ .

## 4.1 Graph coloring



- For a graph  $G$  denote by  $\chi(G)$  the least  $k \in \mathbb{N}$  such that  $G$  is  $k$ -colorable.

**Definition 4.1.1**  $k\text{-COL} := \{G : \chi(G) \leq k\}$

It is well known that  $2\text{-COL} \in \mathbf{P}$ .

**Theorem 4.1.2**  $3\text{-COL} \in \mathbf{NPC}$ .

- The proof that  $3\text{-COL}$  is in  $\mathbf{NP}$  is left as homework

## 4.2 NEA- $k$ -CNF-SAT

- Let  $\varphi$  be a formula

## 4.2 NEA- $k$ -CNF-SAT

- Let  $\varphi$  be a formula
- $\varphi$  is said to be *not all equal satisfiable* (NAE-SAT) if it has a satisfying assignment such that in each clause it has at least one satisfied literal and at least one that is not satisfied.

## 4.2 NAE- $k$ -CNF-SAT

- Let  $\varphi$  be a formula
- $\varphi$  is said to be *not all equal satisfiable* (NAE-SAT) if it has a satisfying assignment such that in each clause it has at least one satisfied literal and at least one that is not satisfied.

### Definition 4.2.1 (NAE- $k$ -CNF-SAT)

NAE- $k$ -CNF-SAT  $:= \{\varphi : \varphi \text{ is NAE-SAT, with exactly } k \text{ literals in each clause}\}.$

## 4.2 NAE- $k$ -CNF-SAT

- Let  $\varphi$  be a formula
- $\varphi$  is said to be *not all equal satisfiable* (NAE-SAT) if it has a satisfying assignment such that in each clause it has at least one satisfied literal and at least one that is not satisfied.

### Definition 4.2.1 (NAE- $k$ -CNF-SAT)

NAE- $k$ -CNF-SAT  $:= \{\varphi : \varphi \text{ is NAE-SAT, with exactly } k \text{ literals in each clause}\}.$

- We are going to show

$$\text{NAE-}k\text{-CNF-SAT} \leq_p \text{3-COL}.$$

### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:

### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. Start with a single vertex  $D$ . This is our *Don't care vertex*.



### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. Start with a single vertex  $D$ . This is our *Don't care vertex*.
  2. For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$ , add an edge between them, and connect both to  $D$ . This are our *variable gadgets*.

### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. Start with a single vertex  $D$ . This is our *Don't care vertex*.
  2. For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$ , add an edge between them, and connect both to  $D$ . This are our *variable gadgets*.
  3. For each clause  $l_1 \vee l_2 \vee l_3$  we create a triangle with 3 vertices named  $l_1, l_2, l_3$ , this is our *clause gadget*.

### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. Start with a single vertex  $D$ . This is our *Don't care vertex*.
  2. For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$ , add an edge between them, and connect both to  $D$ . This are our *variable gadgets*.
  3. For each clause  $l_1 \vee l_2 \vee l_3$  we create a triangle with 3 vertices named  $l_1, l_2, l_3$ , this is our *clause gadget*.
  4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.

### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. Start with a single vertex  $D$ . This is our *Don't care vertex*.
  2. For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$ , add an edge between them, and connect both to  $D$ . This are our *variable gadgets*.
  3. For each clause  $l_1 \vee l_2 \vee l_3$  we create a triangle with 3 vertices named  $l_1, l_2, l_3$ , this is our *clause gadget*.
  4. For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.

•

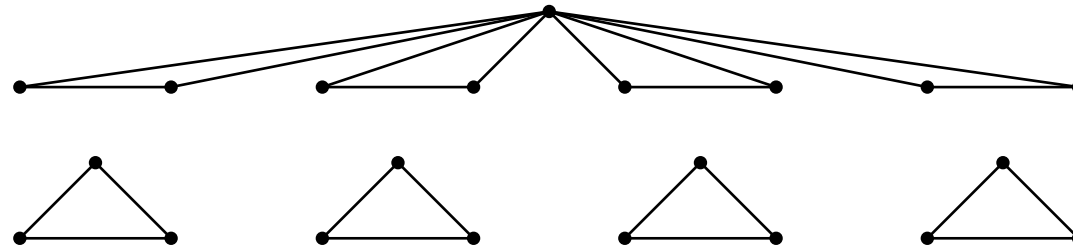
### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  - Start with a single vertex  $D$ . This is our *Don't care vertex*.
  - For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$ , add an edge between them, and connect both to  $D$ . This are our *variable gadgets*.
  - For each clause  $l_1 \vee l_2 \vee l_3$  we create a triangle with 3 vertices named  $l_1, l_2, l_3$ , this is our *clause gadget*.
  - For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.



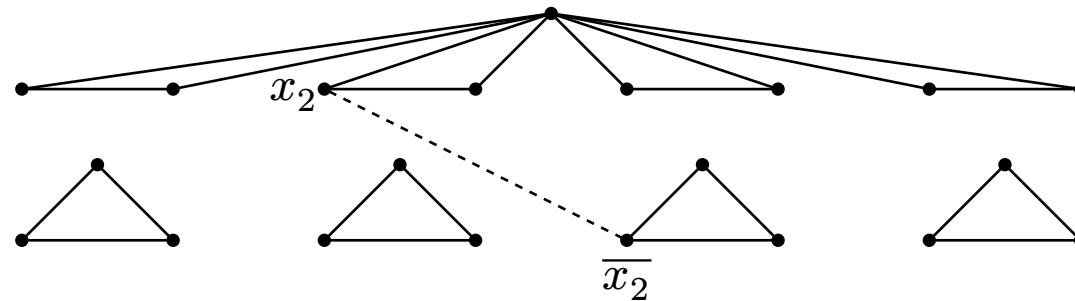
### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  - Start with a single vertex  $D$ . This is our *Don't care vertex*.
  - For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$ , add an edge between them, and connect both to  $D$ . This are our *variable gadgets*.
  - For each clause  $l_1 \vee l_2 \vee l_3$  we create a triangle with 3 vertices named  $l_1, l_2, l_3$ , this is our *clause gadget*.
  - For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.



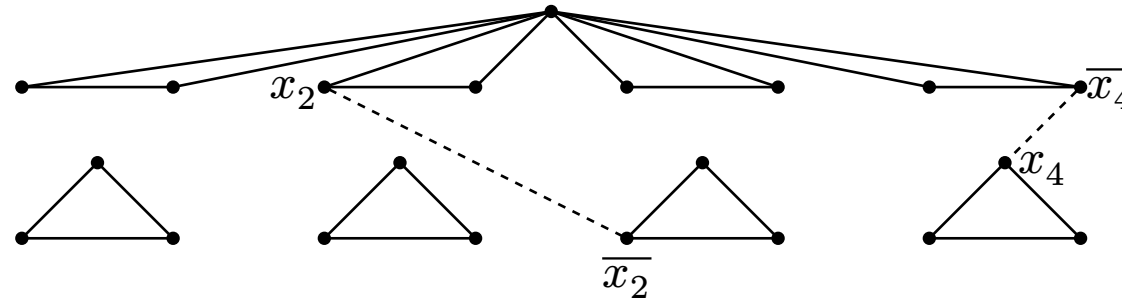
### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  - Start with a single vertex  $D$ . This is our *Don't care vertex*.
  - For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$ , add an edge between them, and connect both to  $D$ . This are our *variable gadgets*.
  - For each clause  $l_1 \vee l_2 \vee l_3$  we create a triangle with 3 vertices named  $l_1, l_2, l_3$ , this is our *clause gadget*.
  - For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.



### 4.3 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  - Start with a single vertex  $D$ . This is our *Don't care vertex*.
  - For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$ , add an edge between them, and connect both to  $D$ . This are our *variable gadgets*.
  - For each clause  $l_1 \vee l_2 \vee l_3$  we create a triangle with 3 vertices named  $l_1, l_2, l_3$ , this is our *clause gadget*.
  - For each literal in the clause gadgets, connect it to the complementary variable from the variable gadget.



- The algorithm runs in poly time(why?)



## 4.4 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- We need to show that

$$\varphi \in \text{NAE-}k\text{-CNF-SAT} \Leftrightarrow G_\varphi \in \text{3-COL}$$

$\Rightarrow$ :

- Given a satisfying NAE assignment  $\alpha_\varphi$  for  $\varphi$ , define the following 3-coloring of  $G_\varphi$ :
  - $D$  will be colored as **D**
  - For each variable  $x_i$ , if  $x_i$  is assigned true under  $\alpha_\varphi$  color  $x_i$  as **T** and  $\overline{x_i}$  in **F**, otherwise color  $x_i$  as **F** and  $\overline{x_i}$  in **T**
  - For each clause gadget, scan the corresponding clause  $c$ , color first literal that is assigned true with **T**, the first that assigned false with **F**, and color the vertex that was left with **D**.
- each edge inside vertex/clause gadgets have both ends in different colors
- W.L.O.G, let  $x$  be a variable assigned true by  $a_\varphi$ , as  $a_\varphi$  is proper, all vertices  $\overline{x}$  are **D** or **F**. So all edges between clause/vertex gadgets have both ends in different colors.

## 4.5 NAE- $k$ -CNF-SAT $\leq_p$ 3-COL

- We need to show that

$$\varphi \in \text{NAE-}k\text{-CNF-SAT} \Leftrightarrow G_\varphi \in \text{3-COL}$$

$\Leftarrow$ :

- Given a 3-coloring  $\psi$  of  $G_\varphi$ , we define a NAE-satisfying assignment for  $\varphi$ .
- As all of the variable gadgets form a triangles with a common vertex  $D$ , it leaves them with two colors to be chosen.
- Take the variable gadget for an arbitrary variable  $x$  and set the color under  $x$  to be true and the color under  $\bar{x}$  to be false. This defines a valid assignment to the variables of  $\varphi$ .
- The assignment is NAE as each clause gadget has one variable colored true and one false.

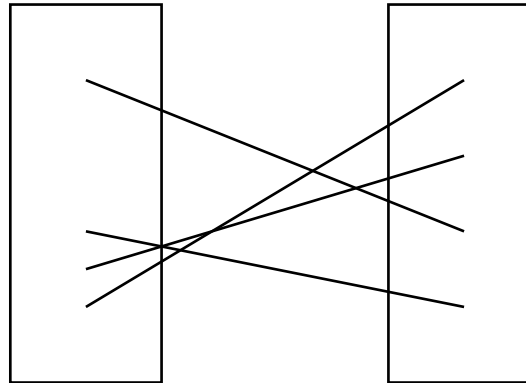
## 4.6 Max-cut

- Given a graph  $G$ , a *cut* is defined as the set of edges between  $S \subseteq V(G)$  and  $\bar{S} = V(G) \setminus S$ .
- We denote the set of edges by

$$E_G(S, \bar{S}) := \{(u, v) : (u, v) \in E(G), u \in S, v \in \bar{S}\}$$

and the number of edges by

$$e_G(S, \bar{S}) := |E_G(S, \bar{S})|$$



## 4.6 Max-cut

Denote by  $\sigma(G) := \max_{S \subseteq V(G)} e_G(S, \overline{S})$ .

**Definition 4.6.1 (MAX-CUT)**  $\text{MAX-CUT} := \{ \langle G, k \rangle : \sigma(G) \geq k \}$

**Theorem 4.6.2**  $\text{MAX-CUT} \in \text{NPC}$

- The proof that MAX-CUT is in **NP** is left as homework.

## 4.7 Max-cut

- We are going to show

$$\text{NAE-}k\text{-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:

## 4.7 Max-cut

- We are going to show

$$\text{NAE-}k\text{-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$  and an edge between them. These are our *variable gadgets*.

## 4.7 Max-cut

- We are going to show

$$\text{NAE-}k\text{-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$  and an edge between them. These are our *variable gadgets*.
  2. For each clause  $l_1 \vee l_2 \vee l_3$ , we create a triangle with vertices  $l_1, l_2, l_3$ ; this is our *clause gadget*.

## 4.7 Max-cut

- We are going to show

$$\text{NAE-}k\text{-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$  and an edge between them. These are our *variable gadgets*.
  2. For each clause  $l_1 \vee l_2 \vee l_3$ , we create a triangle with vertices  $l_1, l_2, l_3$ ; this is our *clause gadget*.
  3. For each literal vertex in a clause gadget, connect it to the complementary literal vertex in the variable gadget.



## 4.7 Max-cut

- We are going to show

$$\text{NAE-}k\text{-CNF-SAT} \leq_p \text{MAX-CUT}.$$

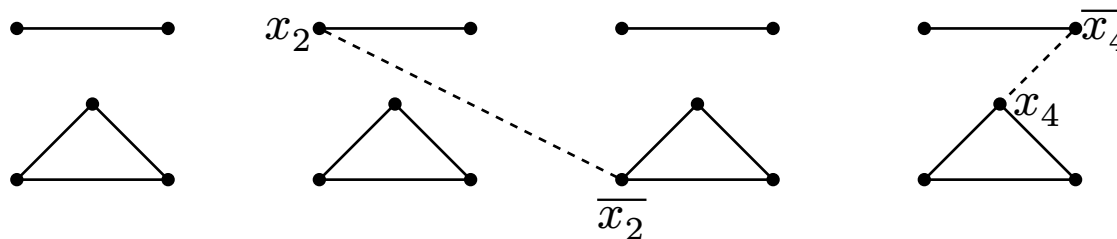
- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  1. For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$  and an edge between them. These are our *variable gadgets*.
  2. For each clause  $l_1 \vee l_2 \vee l_3$ , we create a triangle with vertices  $l_1, l_2, l_3$ ; this is our *clause gadget*.
  3. For each literal vertex in a clause gadget, connect it to the complementary literal vertex in the variable gadget.
- We return the pair  $\langle G_\varphi, n + 5m \rangle$ .

## 4.7 Max-cut

- We are going to show

$$\text{NAE-}k\text{-CNF-SAT} \leq_p \text{MAX-CUT}.$$

- Given a 3-CNF formula  $\varphi$ , define  $G_\varphi$  as follows:
  - For each variable  $x_i$  of  $\varphi$ , add two new vertices  $x_i, \overline{x_i}$  and an edge between them. These are our *variable gadgets*.
  - For each clause  $l_1 \vee l_2 \vee l_3$ , we create a triangle with vertices  $l_1, l_2, l_3$ ; this is our *clause gadget*.
  - For each literal vertex in a clause gadget, connect it to the complementary literal vertex in the variable gadget.
- We return the pair  $\langle G_\varphi, n + 5m \rangle$ .



- The algorithm runs in poly time(why?)

## 4.8 Max-cut

- We need to show that

$$\varphi \in \text{NAE-}k\text{-CNF-SAT} \Leftrightarrow G_\varphi \in \text{MAX-CUT}$$

$\Rightarrow$ :

- Given a satisfying NAE assignment  $\alpha_\varphi$  for  $\varphi$ , define  $S \subseteq V(G_\varphi)$  to consist of all vertices whose label is a literal assigned true under  $\alpha_\varphi$ .
- As  $\alpha_\varphi$  is consistent, all variable gadgets must cross  $(S, \bar{S})$  adding  $n$  edges to the cut.
- As  $\alpha_\varphi$  is a valid NAE assignment, at least two edges cross  $(S, \bar{S})$  in every clause gadget, adding  $2m$  edges to the cut.
- Each edge between a variable and clause gadget has the form  $(l, \bar{l})$ , which means it is also in the cut, adding  $3m$  edges to the cut.
- Overall, we count at least  $n + 5m$  edges.

## 4.9 Max-cut

- We need to show that

$$\varphi \in \text{NAE-}k\text{-CNF-SAT} \Leftrightarrow G_\varphi \in \text{MAX-CUT}$$

$\Leftarrow$ :

- Suppose that  $\langle G_\varphi, n + 5m \rangle \in \text{MAX-CUT}$ . Let  $(S, \bar{S})$  be a cut of  $G_\varphi$  such that  $e_{G_\varphi}(S, \bar{S}) = n + 5m$ .
- Define the assignment  $\alpha_\varphi$  for  $\varphi$  in which all variable gadget literals found in  $S$  are assigned true and all remaining are assigned false. This defines a consistent assignment.
- It remains to prove that  $\alpha_\varphi$  is NAE-satisfying.
  - Every edge between a variable and clause gadget is also in the cut and has the form  $(l, \bar{l})$ .
  - Because  $e_{G_\varphi}(S, \bar{S}) = n + 5m$ , each clause gadget has 2 edges crossing  $(S, \bar{S})$ , meaning each gadget has at least one vertex in  $S$  and one in  $\bar{S}$ .
  - Take a vertex  $l$  in a clause gadget that is in  $S$ . Then the edge  $(l, \bar{l})$  implies that  $\bar{l} \in \bar{S}$ , meaning the literal corresponding to  $l$  is assigned true. In a similar manner, if
  - $l$  is in  $\bar{S}$  then the edge  $(l, \bar{l})$  means the  $\bar{l} \in S$  implying that  $l$  is assigned false.